

**Автоматизация откликов на
платформе HeadHunter (HH
AutoApply System)
Техническое задание
Листов «7»**

Казань 2026

Оглавление

АННОТАЦИЯ	3
1 ВВЕДЕНИЕ	3
2 Основания для разработки	3
3 Назначение разработки	3
4 Требования к Программному Комплексу	4
5 Требования к программной документации	6
6 Техничко-экономические показатели	6
7 Стадии и этапы разработки	6
8 Порядок контроля и приемки	7
9 Примечания	7

АННОТАЦИЯ

Данное техническое задание составлено на разработку Программного Комплекса (ПК) «НН AutoApply». Данная система будет построена на архитектуре микросервисов и содержать следующий функционал:

1. Модуль авторизации, управления сессиями и парсинга вакансий;
 2. Модуль автоматизированной подачи откликов;
 3. Модуль генерации контента на базе искусственного интеллекта (AI).
- Система предназначена для автоматизации рутинных процессов поиска работы, анализа рынка труда и персонализации сопроводительных материалов.

1 ВВЕДЕНИЕ

Настоящее техническое задание распространяется на разработку программы «НН AutoApply», используемой для автоматизации взаимодействия соискателя с платформой HeadHunter.

Цели, задачи:

- Снижение временных затрат на поиск вакансий и подачу откликов.
- Повышение качества сопроводительных писем за счет использования AI.
- Предоставление аналитики рынка труда в реальном времени.

Программа является актуальной, так как:

1. Рынок труда требует высокой скорости реакции на новые вакансии.
2. Ручная подготовка уникальных откликов для каждой вакансии является трудоемким процессом.

2 Основания для разработки

«НН AutoApply» разрабатывается в соответствии с:

- Внутренним приказом о начале НИОКР.
- Требованиями к автоматизации процессов HR-технологий.

3 Назначение разработки

Основное назначение «НН AutoApply» заключается в автоматическом поиске подходящих вакансий по заданным критериям, генерации персонализированных сопроводительных писем и массовой подаче откликов в безопасном режиме, имитирующем действия пользователя.

4 Требования к Программному Комплексу

4.1 Требования к функциональным характеристикам

4.1.1 Выполняемые функции

4.1.1.1 Для пользователя (через интерфейс управления/бота):

- Авторизация: Безопасный вход в аккаунт HH.ru через OTP-код.
- Настройка поиска: Ввод ключевых слов, настройка умных фильтров (зарплата, удалёнка, исключения).
- Управление откликами: Установка расписания автооткликов, лимитов (до 100 за сессию).
- Работа с резюме: Загрузка PDF/текста для «Про жарки» (AI-анализа).
- Аналитика: Просмотр статистики (вилки зарплат, топ работодателей, распределение по форматам).
- История: Экспорт истории откликов в CSV.
- Настройки AI: Выбор модели для генерации текста.

4.1.1.2 Системные функции (Микросервисы):

- Сервис Авторизации и Сбора (Auth & Search Service):
 - Хранение и ротация сессионных токенов.
 - Парсинг вакансий по запросу.
 - Фильтрация по «Черному списку» компаний.
 - Уведомления о новых вакансиях (таймер раз в день).
- Сервис Откликов (Apply Service):
 - Автоматическая подача отклика на вакансию.
 - Соблюдение лимитов и задержек (anti-detect).
 - Ведение лога отправленных откликов.
- Сервис Контента (AI Content Service):
 - Генерация уникального сопроводительного письма под описание вакансии.
 - Анализ резюме (критика, рекомендации).
 - Взаимодействие с внешними AI-моделями (LLM).

4.1.2 Исходные данные:

Вводятся пользователем: поисковые запросы, параметры фильтров, текст резюме, OTP-код.

Получаются автоматически: описание вакансий, требования работодателей, сессионные токены.

4.1.3 Результаты

- Сформированные отклики на платформе HH.ru.
- Сгенерированные тексты сопроводительных писем.
- Аналитика в виде графиков

4.2 Требования к надежности

4.2.1 Предусмотреть контроль вводимой информации (валидация форматов, проверка обязательных полей).

4.2.2 Предусмотреть защиту от некорректных действий пользователя (блокировка кнопок при отсутствии сессии).

4.2.3 Предусмотреть механизм повторных попыток при ошибках сети или API.

4.2.4 Реализовать защиту от блокировок со стороны платформы (рандомизация интервалов запросов).

4.3 Условия эксплуатации

4.3.1 Условия эксплуатации в соответствии с СанПин 2.2.2.542 — 96 (для рабочих мест операторов) и современными нормами эксплуатации серверного оборудования.

4.3.2 Система должна быть доступна 24/7 в режиме фоновой работы.

4.4 Требования к составу и параметрам технических средств

4.4.1 Программное обеспечение должно функционировать на серверах с архитектурой x86-64 или ARM64 (контейнеризация Docker).

4.4.2 Минимальная конфигурация технических средств (для развертывания):

4.4.2.1 Тип процессора: 2 vCPU (современный аналог).

4.4.2.2 Объем ОЗУ: 4 Гб (для базовой работы), 8 Гб (рекомендуется для AI-модуля).

4.4.2.3 Дисковое пространство: 20 Гб SSD.

4.5 Требования к информационной и программной совместимости

4.5.1 Программное обеспечение должно работать под управлением ОС Linux (Ubuntu 20.04+) или в облачных средах (Kubernetes).

4.5.2 Входные данные должны быть представлены в виде JSON/API запросов или форм в интерфейсе.

4.5.3 Результаты должны быть представлены в следующем формате:

Статус «одобreno/отправлено» в системе.

Файлы CSV/PDF для скачивания.

4.5.4 Программное обеспечение разрабатывается на языках: Go, Java Python (FastAPI/Asyncio). База данных: PostgreSQL/Redis.

4.5.5 Интеграция с внешними AI-провайдерами через REST API.

4.6 Требования к маркировке и упаковке

Требования к маркировке и упаковке не предъявляются (программный продукт распространяется цифровым способом).

4.7 Требования к транспортированию и хранению

Требования к транспортировке и хранению не предъявляются. Резервное копирование данных осуществляется на выделенном хранилище.

4.8 Специальные требования

- Обеспечить шифрование чувствительных данных (токены сессий, логины) в базе данных.
- Реализовать возможность выбора AI-модели пользователем.

5 Требования к программной документации

5.1 Разрабатываемые программные модули должны быть самодокументированы, т.е. тексты программ должны содержать все необходимые комментарии (Docstrings).

5.2 В состав сопровождающей документации должны входить:

5.2.1 Техническое задание.

5.2.2 Инструкция пользователя (как пройти авторизацию).

5.2.3 Схема архитектуры микросервисов.

6 Техничко-экономические показатели

Техничко-экономическое обоснование разработки не выполняется (в рамках данного документа). *Примечание:* Ожидаемый эффект заключается в экономии до 20 часов рабочего времени соискателя в неделю.

7 Стадии и этапы разработки

№	Название этапа	Срок, дни	Отчётность
1	Проектирование		Разработка архитектуры микросервисов, проектирование БД
2	Разработка MVP		Реализация авторизации и простого отклика
3	Интеграция AI:		Подключение модуля генерации писем и анализа резюме.

4	Тестирование		Проверка на устойчивость к блокировкам, нагрузочное тестирование
5	Внедрение		Развертывание на боевом сервере

8 Порядок контроля и приемки

8.1 Порядок контроля

Контроль выполнения осуществляется заказчиком путем тестирования функционала на тестовом аккаунте HH.ru.

8.2 Критерии приемки

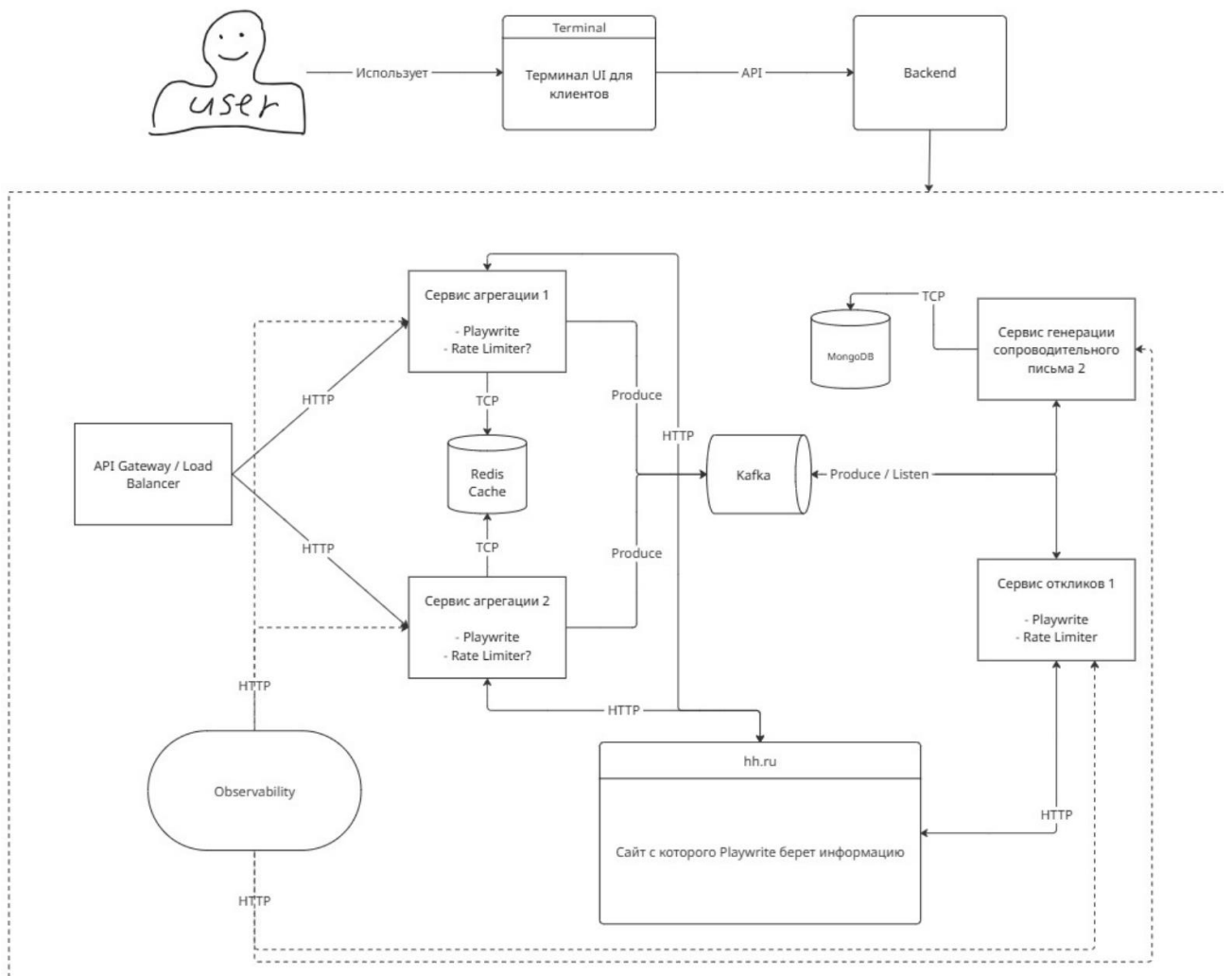
- Успешная авторизация по ОТП.
- Корректная генерация письма для 10 различных вакансий.
- Успешный отклик на вакансию

9 Примечания

10.1 В процессе выполнения работы возможно уточнение отдельных требований технического задания по взаимному согласованию руководителя и исполнителя, особенно в части интеграции с внешними API-сервисами и изменениями API платформы HeadHunter.

10.2 Разработка ведется с учетом соблюдения условий использования сторонних сервисов.

C4 + Sequence



1.User Stories

Описывают потребности пользователя в формате: «Как [роль], я хочу [действие], чтобы [ценность]».

ID	Роль	Story	Ценность
US-01	Пользователь	Как пользователь, я хочу авторизоваться через OTP-код, чтобы безопасно подключить свой аккаунт HH.ru к системе без передачи пароля.	Безопасность данных.
US-02	Пользователь	Как пользователь, я хочу настроить фильтры поиска (зарплата, удаленка, черный список), чтобы система игнорировала неподходящие вакансии.	Экономия времени на фильтрацию.
US-03	Пользователь	Как соискатель, я хочу, чтобы система автоматически отправляла отклики (до 100 за сессию), чтобы повысить вероятность приглашения на собеседование.	Автоматизация рутины.
US-04	Пользователь	Как соискатель, я хочу получать уникальные сопроводительные письма от AI под каждую вакансию, чтобы выделиться среди других кандидатов.	Персонализация отклика.
US-05	Пользователь	Как соискатель, я хочу загрузить резюме для AI-анализа («прожарки»), чтобы	Повышение качества резюме.

		получить рекомендации по его улучшению.	
US-06	Пользователь	Как соискатель, я хочу видеть аналитику (вилки зарплат, топ работодателей), чтобы понимать ситуацию на рынке труда.	Информированность.
US-07	Пользователь	Как соискатель, я хочу выгружать историю откликов в CSV, чтобы вести личный учет отправленных заявок.	Контроль и отчетность.

2.Use Cases

Детальное описание сценариев взаимодействия.

UC-01: Автоматическая подача отклика (Auto-Apply)

Участники: Пользователь (инициатор), Система (исполнитель), Сервис AI, Платформа HH.ru.

Предусловия: Пользователь авторизован, настройки поиска сохранены, лимиты откликов не исчерпаны.

Основной поток :

1. Сервис авторизации и сбора находит новую вакансию по заданным критериям.
2. Система проверяет компанию по «Черному списку». Если компания в списке — переход к шагу 1 (следующая вакансия).
3. Сервис Контента (AI) получает описание вакансии и текст резюме пользователя.
4. AI генерирует уникальное сопроводительное письмо.
5. Сервис откликов имитирует действия пользователя (соблюдая задержки/anti-detect).
6. Система отправляет отклик с сгенерированным письмом на HH.ru.
7. Система записывает результат в лог и обновляет статистику.

Альтернативный поток (Ошибки):

- **4а. Ошибка генерации AI:** Если AI не ответил за 10 сек, система использует шаблонное письмо или пропускает вакансию.
 - **6а. Блокировка со стороны HH:** Если получен код ошибки (403/429), система приостанавливает работу на случайный интервал времени (рандомизация).
-

UC-02: AI-анализ резюме («Прожарка»)

Участники: Пользователь, Сервис Контента.

Предусловия: Пользователь загрузил файл резюме (PDF/TXT).

Основной поток:

1. Пользователь нажимает кнопку «Анализ резюме».
2. Система извлекает текст из файла.
3. Система отправляет текст в AI-модель с промптом на критику и рекомендации.
4. AI возвращает структурированный отчет (плюсы, минусы, советы).
5. Система отображает отчет пользователю в интерфейсе.

3.NFR

Требования к качеству системы

Производительность и Надежность

- **Доступность:** Система должна работать в режиме 24/7 (фоновый режим).
- **Масштабируемость:** Архитектура должна поддерживать развертывание в Docker/Kubernetes (микросервисы).
- **Отказоустойчивость:** Реализовать механизм повторных попыток (retry) при ошибках сети или API HH.ru.
- **Ресурсы:** Минимальные требования к серверу: 2 vCPU, 4 Гб ОЗУ (базовый режим), 8 Гб ОЗУ (с активным AI-модулем).

Безопасность (Security)

- **Аутентификация:** Вход только через OTP-код (без хранения паролей).
- **Шифрование:** Чувствительные данные (сессионные токены, логины) должны быть зашифрованы в БД.

- **Anti-Detect:** Реализовать рандомизацию интервалов запросов и задержек для имитации поведения человека и защиты от бана аккаунта.
- **Валидация:** Контроль вводимых данных и блокировка действий при отсутствии активной сессии.

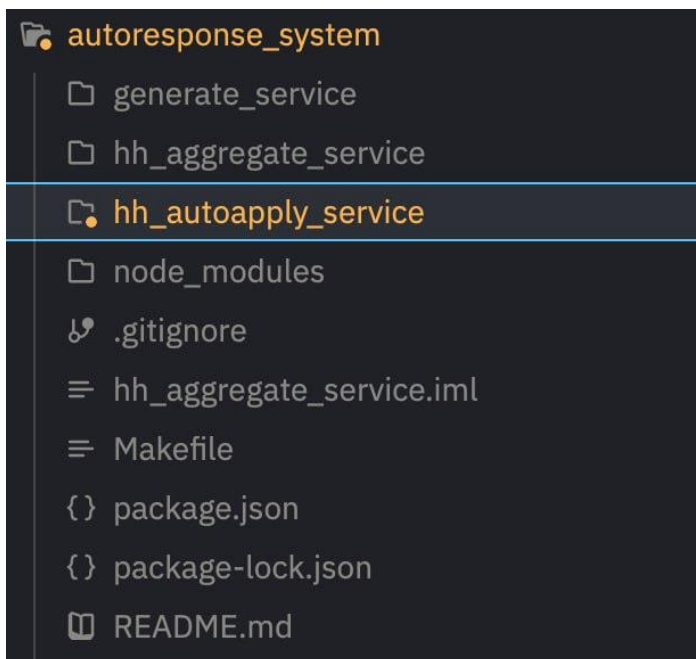
Совместимость и Интерфейсы

- **ОС:** Поддержка Linux (Ubuntu 20.04+) или облачных сред.
- **Форматы данных:** Вход/Выход через JSON API; экспорт отчетов в CSV/PDF.
- **Стек:** Backend на Go/Java/Python (FastAPI); БД PostgreSQL + кэш Redis.
- **Интеграция:** Взаимодействие с внешними AI-провайдерами через REST API.

Документация

- Код должен быть самодокументирован (Docstrings).
- Обязательное наличие схемы архитектуры микросервисов и инструкции пользователя.

Отчет о микросервисах



“generate_service” – Хусаенов Роберт

“hh_aggregate_service” – Ежиков Кирилл

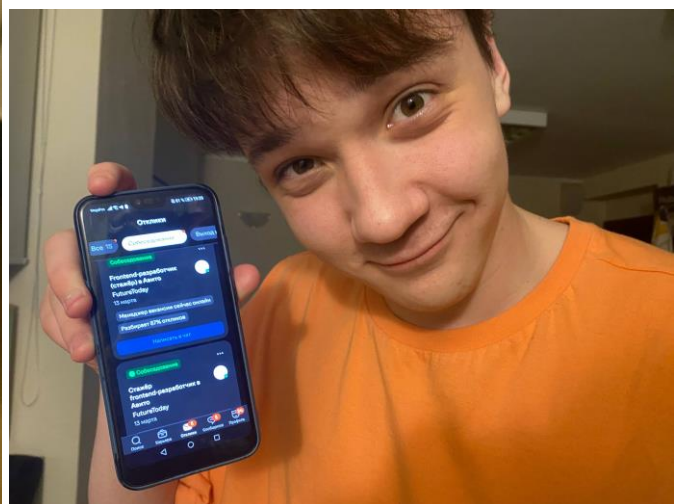
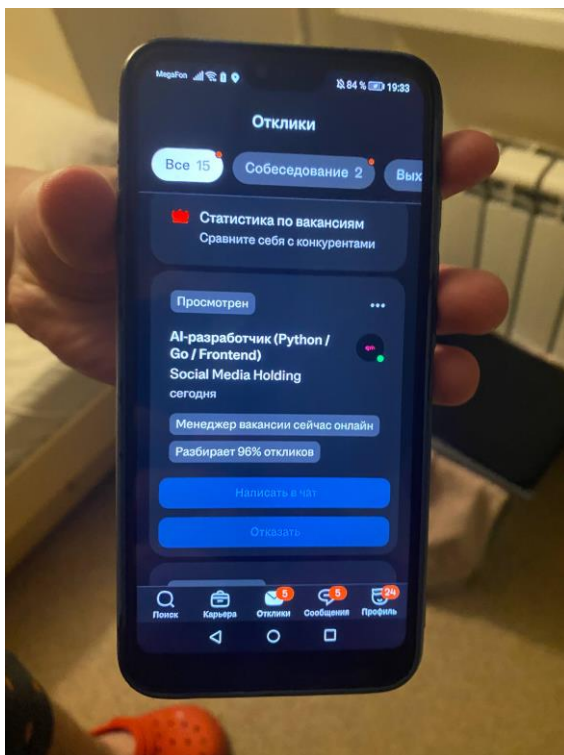
“hh_autoapply_service” – Тарасов Тимофей

```

2026/03/15 19:31:02 Starting auto-apply process for request 61, user 2
2026/03/15 19:31:02 Getting HH token from Java service for user 2
2026/03/15 19:31:03 Got HH token from Java service for user 2
2026/03/15 19:31:03 Got HH token (storageState) from Java service for user 2, length: 626080
2026/03/15 19:31:03 Getting vacancies from Kafka topic: vacancies.parsed
2026/03/15 19:31:03 Waiting for vacancies from Kafka (max wait: 5m0s, idle timeout: 10s)
2026/03/15 19:31:39 Received batch of 20 vacancies from Kafka (total: 20)
[{"130667533 AI-разработчик (Python / Go / Frontend) Social Media Holding https://kazan.hh.ru/vacancy/130667533?query=Go+Developer&httpFrom=vacancy_search_list Кто мы: Мы создаем экосистему сервисов и AI-продуктов, которая уже работает в десятках стран. Наш код двигает миллионы транзакций в день, соединяя людей, системы и данные. Мы растем кратно каждый год и ищем не просто сотрудников – будущих партнеров, которые живут своей работой и хотят влиять на будущее. Принцип простой: Чем успешнее твой проект – тем выше твой доход. Ты не делаешь стартап “в никуда”. Ты получаешь конкретный проект, который после реализации гарантированно зарабатывает деньги. Твоя задача – довести его до идеала и зарабатывать вместе с ним. Кого мы ищем: Мы ищем разработчиков нового поколения – тех, кто уже работает с ИИ или осознанно готов перейти на новый формат разработки. В этой роли ты будешь: Разрабатывать новые сервисы и улучшать существующие продукты; Реализовывать фичи от идеи до деплоя; Работать с высокими нагрузками и большими объемами данных; Оптимизировать системы и находить узкие места. Нам подходят два типа кандидатов: Ты уже работаешь с ИИ: 50%+ кода пишешь с помощью Claude Code, GPT-5 или других моделей; Строишь собственные AI-пайплайны и автоматизируешь рутину; Умеешь писать код без ИИ, когда это необходимо. Ты готов перестроиться: Понимаешь, что классический подход к программированию устаревает; Хочешь кратно увеличить эффективность за счёт AI; Готов быстро войти в новый формат работы и реальные проекты. Обязательное требование (без исключений): Практический опыт работы с MCP (Model Context Protocol). Мы предлагаем: Формат работы: гибрид в Москве или 2-3 месяца удаленки, далее переезд в Москву (релокация с нас); В московском офисе – PlayStation, вкусный китайский чай и живая атмосфера, но можно работать и из любой точки мира; Гибкий график: важны результаты и ответственность, а не строгие рамки; Прозрачная система вознаграждения: фикс + бонусы за результат + премии за креативные решения; Совместные выезды в тёплые страны, тимбилдинги, корпоративы и спортивные выходные всей командой; Участие в международных выставках и митапах (Дубай, Ереван, Москва и другие); Возможность возмещения тренажерного зала, курсов; Мы ценим инициативу: можешь оптимизировать

```

Рисунок 2 – Успешное получение распаршенных вакансий в сервисе `hh_autoapply_service` из сервиса `hh_aggregate_service` (по кафке)



Рисунки 3, 4 – демонстрация откликов и первое приглашение на собеседование

```
def build_prompt(data: dict) -> str:
    return f"""
Сгенерируй ОЧЕНЬ короткое сопроводительное письмо для отклика на вакансию.

Входные данные:

Название вакансии: "{data['title']}"
Компания: "{data['company']}"
Описание/требования: "{data['requirements']}"

Жёсткие требования:

2-3 предложения, не больше
Без приветствий и подписей
Без фраз: «с большим интересом», «уверен», «буду рад», «внести вклад»
Текст должен выглядеть как написанный человеком, не HR и не нейросетью
Прямо укажи, что есть релевантный опыт по вакансии {data['title']}
Профессионально, но разговорно
Только финальный текст письма
Никаких комментариев, пояснений или советов
Русский язык
не оставляй в конце системный комментарий с [Ваше имя]
"""
```

Рисунок 5 – Метод в сервисе **generate_service** , который делает промт для сопроводительного письма

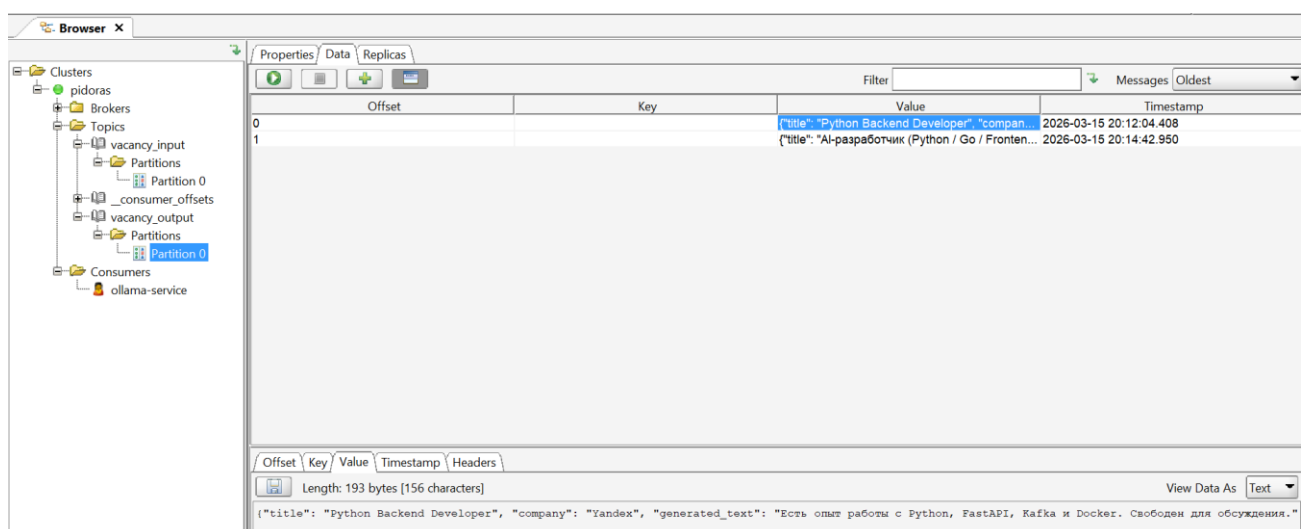


Рисунок 6

```
PS C:\Users\husae\OneDrive\Desktop\Обучение> & C:/Users/husae/AppData/Local/Programs/Python/Python311/python.exe c:/Users/husae/OneDrive/Desktop/Обучение/APS_project/services/generate.py
Service started. Waiting Kafka messages...
Received: {'title': 'Python Backend Developer', 'company': 'Yandex', 'requirements': 'Python, FastAPI, Kafka, Docker'}
Sent result to Kafka
Received: {'title': 'AI-разработчик (Python / Go / Frontend)', 'company': 'Social Media Holding', 'requirements': 'Кто мы: Мы создаем экосистему сервисов и AI-продуктов, которая уже работает в десятках стран. Наш код двигает миллионы транзакций в день, соединяя людей, системы и данные. Мы растем кратно каждый год и ищем не просто сотрудников – лучших партнеров, которые живут своей работой и хотят влиять на будущее. Принцип простой: Чем успешнее твой проект – тем выше твой доход. Ты не делаешь стартап “в никуда”. Ты получаешь конкретный проект, который после реализации гарантированно зарабатывает деньги. Твоя задача – довести его до идеала и зарабатывать вместе с ним. Кого мы ищем: Мы ищем роли ты будешь: Разрабатывать новые сервисы и улучшать существующие продукты; Реализовывать фичи от идеи до деплоя; Работать с высокими нагрузками и большими объемами данных; Оптимизировать системы и находить узкие места. Нам подходят два типа кандидатов: Ты уже работаешь с ИИ: 50%+ кода пишешь с помощью Claude Code, GPT-5 или других моделей; С
```

Рисунок 7

Рисунки 6,7 - Демонстрация работы **generate_service** с kafka

Ссылка на репозиторий: https://github.com/ForpoS7/autoresponse_system

1. Общее описание

HH AutoResponse System — это распределённая микросервисная система для автоматического парсинга вакансий с сайта HH.ru и автоматической подачи откликов на них. Система предназначена для соискателей, которые хотят автоматизировать процесс поиска работы и отправки резюме.

Архитектура системы

Проект состоит из трёх независимых сервисов, которые взаимодействуют через общую инфраструктуру:

Сервисы: порт 8080

1. **hh_aggregate_service (Java/Spring Boot)** —
2. **hh_autoapply_service (Go)** — порт 8081
3. **generate_service (Python)** — без HTTP, работает через Kafka

Инфраструктура:

- **PostgreSQL (порт 5444)** — хранение пользователей, токенов, вакансий, заявок на автоотклик
- **Apache Kafka (порт 9092)** — асинхронная передача сообщений между сервисами
- **Ollama (порт 11434)** — локальная LLM для генерации сопроводительных писем

Технологический стек

Backend:

- **Java 17+ co Spring Boot 3.2.5** — основной сервис парсинга
- **Go 1.22+ c Gorilla Mux** — сервис автооткликов
- **Python 3.x** — сервис генерации сопроводительных писем

Базы данных и брокеры:

- **PostgreSQL 16** — реляционная база данных
- **Apache Kafka 3.7.0** — брокер сообщений

Автоматизация браузера:

- **Playwright 1.58.0** — используется в Java и Go сервисах для парсинга HH.ru

AI/ML:

- **Ollama с моделью Qwen2.5:7b** — генерация текста сопроводительных писем

Инструменты сборки:

- **Gradle 8.x** — для Java сервиса
- **Go modules** — для Go сервиса

2. ФУНКЦИОНАЛ СЕРВИСОВ

2.1 HH Aggregate Service (Java)

Основное назначение: Парсинг вакансий с HH.ru и публикация их в Kafka.

Ключевой функционал:

1. Аутентификация и авторизация

- Регистрация новых пользователей через POST /api/auth/register
- Логин через POST /api/auth/login
- Выдача JWT-токенов с временем жизни 24 часа
- Защита эндпоинтов через Spring Security + JWT фильтр

2. Парсинг вакансий

- GET /api/vacancies?query={текст}&page={номер}
- Автоматический переход на страницу поиска HH.ru
- Извлечение данных: ID, название, работодатель, URL, описание, зарплата, валюта, регион
- Загрузка полного описания вакансии с отдельной страницы
- Публикация распарсенных вакансий в Kafka топик vacancies.parsed

3. Управление токеном HH.ru

- POST /api/hh-token — извлечение токена сессии из cookies HH.ru через браузер
- GET /api/hh-token — получение сохранённого токена из базы данных
- Токен необходим для авторизованного доступа к HH.ru

4. Планировщик задач

- GET /api/scheduler/config — получение конфигурации планировщика
- Поддержка cron-задач для автоматического парсинга (в текущей версии отключено)

Ход работы сервиса:

1. Пользователь регистрируется или логинится → получает JWT токен
2. Пользователь отправляет запрос на парсинг вакансий с указанием поискового запроса
3. Сервис через Playwright открывает браузер, переходит на HH.ru по сформированному URL
4. Парсится список вакансий со страницы поиска (элементы с data-qa="vacancy-serp__vacancy")

5. Для каждой вакансии отдельно загружается страница и извлекается полное описание
6. Распарсенные вакансии публикуются в Kafka топик vacancies.parsed
7. Вакансии также сохраняются в PostgreSQL для истории

Технические особенности:

- Используется Playwright для эмуляции реального браузера (обход защиты от ботов)
- Парсинг происходит последовательно (не параллельно) для стабильности
- HTML очищается через JSoup от лишних тегов
- JWT токены хранятся в базе данных с привязкой к пользователю

2.2 HH AutoApply Service (Go)

Основное назначение: Автоматическая подача откликов на вакансии с использованием Playwright и генерацией сопроводительных писем через AI.

Ключевой функционал:

1. Аутентификация и авторизация

- Регистрация и логин пользователей (аналогично Java сервису)
- JWT токены с использованием библиотеки golang-jwt/jwt/v5
- Middleware для проверки токенов на всех защищённых эндпоинтах

2. Парсинг вакансий

- GET /api/vacancies?query={текст}&page={номер}
- Аналогичный функционал парсинга через Playwright
- Публикация в Kafka vacancies.parsed

3. Автоотклики

- POST /api/autoapply — создание заявки на автоотклик (параметры: query, apply_count)
- GET /api/autoapply/{id} — получение статуса заявки (статус, количество откликов)

- Автоматический переход на страницу вакансии
- Заполнение формы отклика
- Прикрепление сгенерированного сопроводительного письма
- Логирование результатов (успех/ошибка)

4. Генерация сопроводительных писем

- Интеграция с generate_service через Kafka
- Отправка данных вакансии в топик vacancy_input
- Получение сгенерированного письма из топика vacancy_output
- Письма генерируются через Ollama (модель Qwen2.5:7b)

5. Управление токеном HH.ru

- POST /api/hh-token — извлечение токена из браузера
- GET /api/hh-token — получение сохранённого токена

6. Межсервисное взаимодействие

- HTTP клиент для обращения к Java сервису
- Использование сервисного JWT токена для авторизации между сервисами

Ход работы сервиса:

1. Пользователь создаёт заявку на автоотклик через POST /api/autoapply

- Указывает поисковый запрос (например, "Go Developer")
- Указывает желаемое количество откликов (например, 10)

2. Сервис получает заявку, сохраняет её в базу со статусом "pending"

3. Запускается процесс обработки:

- Через Playwright выполняется поиск вакансий по запросу
- Распарсенные вакансии публикуются в Kafka

- Для каждой вакансии запрашивается генерация сопроводительного письма

4. Генерация письма:

- Данные вакансии (название, компания, требования) отправляются в Kafka vacancy_input
- Generate_service получает сообщение, запрашивает генерацию у Ollama
- Ollama возвращает текст письма (2-3 предложения, без клише)
- Результат публикуется в vacancy_output
- Go сервис получает письмо из Kafka

5. Автоотклик:

- Сервис переходит на страницу вакансии
- Находит кнопку "Откликнуться"
- Заполняет форму (при необходимости)
- Вставляет сгенерированное сопроводительное письмо
- Отправляет отклик
- Результат записывается в лог (успех/ошибка)

6. Статус заявки обновляется:

- applied_count — количество успешных откликов
- status — "pending" → "processing" → "completed" или "failed"

Технические особенности:

- Rate limiting (10 запросов в минуту, burst 5) — защита от блокировки
- Браузер работает в видимом режиме (headless: false) для отладки
- Slow mo 100ms — задержка между действиями для стабильности
- Поддержка межсервисной авторизации через JWT

2.3 Generate Service (Python)

Основное назначение: Генерация коротких сопроводительных писем для вакансий с использованием локальной языковой модели (LLM).

Ключевой функционал:

1. Потребление запросов из Kafka

- Подписка на топик `vacancy_input`
- Group ID: `ollama-service`
- Формат сообщения: JSON с полями `title`, `company`, `requirements`

2. Генерация текста через Ollama

- POST запрос к локальному Ollama API (<http://localhost:11434/api/generate>)
- Модель: `qwen2.5:7b`
- Специальный промпт для генерации коротких писем без клише

3. Публикация результатов в Kafka

- Отправка сгенерированного письма в топик `vacancy_output`
- Формат: JSON с полями `title`, `company`, `generated_text`

Ход работы сервиса:

1. Сервис запускается и подключается к Kafka как консьюмер `vacancy_input`
2. Ожидает сообщения в цикле:

for message in consumer:

data = message.value

3. Получает данные вакансии:

```
{  
  "title": "Go Developer",  
  "company": "Яндекс",  
  "requirements": "Разработка backend-сервисов..."  
}
```

4. Формирует промпт для LLM с жёсткими требованиями:

- 2-3 предложения, не больше
- Без приветствий и подписей
- Без фраз: «с большим интересом», «уверен», «буду рад», «внести вклад»

- Текст должен выглядеть как написанный человеком
- Указание на релевантный опыт
- Профессионально, но разговорно
- Только финальный текст, без комментариев

5. Отправляет запрос к Ollama:

```
response = requests.post(
    "http://localhost:11434/api/generate",
    json={"model": "qwen2.5:7b", "prompt": prompt},
    timeout=120
)
```

6. Получает ответ и публикует в Kafka:

```
{
    "title": "Go Developer",
    "company": "Яндекс",
    "generated_text": "Имею релевантный опыт разработки backend-сервисов..."
}
```

Технические особенности:

- Сервис не имеет HTTP сервера, работает только через Kafka
- Синхронная обработка сообщений (одно за другим)
- Таймаут запроса к Ollama — 120 секунд
- Используется `ensure_ascii=False` для корректной работы с кириллицей
- Логирование через `print()` (для MVP достаточно)

MAKEFILE

Назначение: Централизованное управление всеми сервисами системы через единую точку входа. Makefile позволяет избежать необходимости запоминать множество команд и скриптов для запуска, остановки, тестирования и отладки сервисов.

Доступные команды (targets)

make help

Назначение: Показать справку по всем доступным командам.

Что делает:

- Использует grep для поиска строк с комментарием ##
- Форматирует вывод с выделением имён команд цветом
- Показывает краткое описание каждой команды

make infra

Назначение: Запуск инфраструктуры (PostgreSQL + Kafka) через Docker Compose.

Что делает:

1. Выводит информационное сообщение о запуске инфраструктуры
2. Переходит в директорию hh_aggregate_service
3. Выполняет docker-compose up -d (запуск в фоновом режиме)
4. Выводит сообщение об успешном запуске
5. Ожидает 10 секунд для полного запуска PostgreSQL
6. Показывает статус контейнеров через docker-compose ps

Примечание: После выполнения команды рекомендуется подождать 10-15 секунд перед запуском сервисов для полной инициализации базы данных.

make stop

Назначение: Остановка Docker контейнеров инфраструктуры.

Что делает:

1. Выводит информационное сообщение об остановке
2. Переходит в директорию hh_aggregate_service
3. Выполняет docker-compose down

Важно: Эта команда останавливает только Docker контейнеры. Java и Go сервисы, запущенные через `make java` и `make go`, нужно останавливать отдельно через `Ctrl+C` или команду `kill`

make java

Назначение: Запуск Java сервиса (`hh_aggregate_service`) через Gradle.

Что делает:

1. Выводит информационное сообщение о запуске
2. Переходит в директорию `hh_aggregate_service`
3. Выполняет `./gradlew bootRun` через Gradle wrapper

Особенности:

- Сервис запускается в `foreground` режиме (занимает терминал)
- Для запуска в фоне используйте: `make java &`
- Порт сервиса: 8080
- Время запуска: 15-30 секунд (в зависимости от мощности машины)

make go

Назначение: Запуск Go сервиса (`hh_autoapply_service`) через `go run`.

Что делает:

1. Выводит информационное сообщение о запуске
2. Переходит в директорию `hh_autoapply_service`
3. Выполняет `go run cmd/main.go`

Особенности:

- Сервис запускается в `foreground` режиме
- Для запуска в фоне используйте: `make go &`
- Порт сервиса: 8081
- Время запуска: 3-5 секунд

make build

Назначение: Сборка Go сервиса в бинарный файл.

Что делает:

1. Выводит информационное сообщение о сборке
2. Переходит в директорию hh_autoapply_service
3. Выполняет `go build -o bin/hh_autoapply_service ./cmd/main.go`

Преимущества перед `go run`:

- Быстрее запуск (не требуется компиляция при каждом старте)
- Меньше потребление памяти
- Удобно для деплоя на сервер

make install-playwright

Назначение: Установка браузеров Playwright для Go сервиса.

Что делает:

1. Выводит информационное сообщение об установке
2. Переходит в директорию hh_autoapply_service
3. Выполняет установку браузеров через `playwright-go`

Преимущества использования Makefile

1. Единая точка входа — все команды управления в одном файле
2. Документированность — встроенная справка через `make help`
3. Автоматизация — сокращает количество ручных команд
4. Цветной вывод — наглядная индикация статуса операций
5. Кроссплатформенность — работает на macOS, Linux, Windows (с установленным Make)
6. Согласованность — все члены команды используют одинаковые команды
7. Расширяемость — легко добавить новые команды по мере роста проекта